

TP Architecture 1 :

Prise en main du simulateur

I - Introduction à JSimRisc

L'outil JSimRisc¹ est un outil qui a été développé au sein de la filière EII de l'Enssat (Université de Rennes1). Il a été principalement développé des fins pédagogiques et est basé sur l'architecture présentée dans le livre de Hennessy et Patterson (Computer Architecture : A Quantitative Approach).

L'architecture DLX¹ est une architecture RISC assez proche des processeurs MIPS.

Le gros avantage de cette architecture est que l'on dispose d'un certain nombre de simulateurs pédagogiques qui permettent de mieux comprendre les mécanismes des machines à base de pipeline.

Attention, ce simulateur de processeur ne permet pas d'étudier les processeurs superscalaires.

II - Lancement de JSimRisc

1. Charger le programme JAVA disponible sous le lien suivant :
<https://cairn.enssat.fr/enseignements/Archi/JSimRisc/index.php>
2. Placez vous dans le répertoire JSimRisc ;
3. Tapez JSimRisc la fenêtre de la figure 1 apparaît.

¹ Nous tenons à remercier Daniel Chillet (ENSSAT – Université de Rennes 1) qui nous a mis à disposition cet outil et autorisé à l'utiliser pour la licence Informatique de l'université de la Rochelle

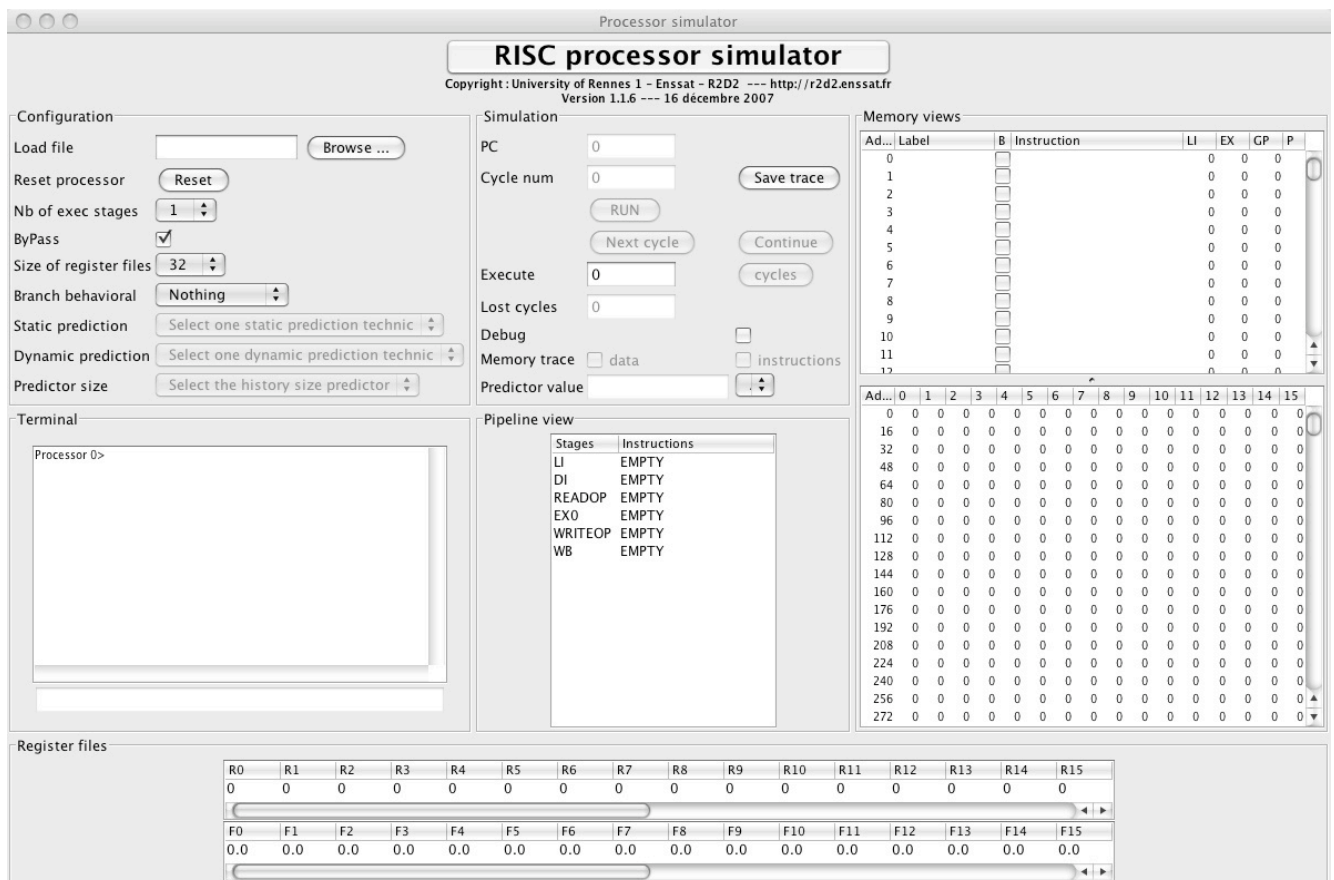


FIGURE 1 – Fenêtre de JSimRisc

La simulation peut être lancée dans plusieurs modes :

- mode run classique, bouton RUN ;
- mode pas à pas : bouton Next cycle ;
- mode break point : bouton Continue.

Le mode RUN lance une exécution et le simulateur stoppe la simulation dès lors que l'instruction exit se trouve dans l'étage pipeline WB.

Le mode break point effectue une simulation du code depuis l'adresse du pointeur de programme courant jusqu'à ce qu'une instruction portant un point d'arrêt entre dans le pipeline, ou jusqu'à la fin du programme. Les points d'arrêt sont positionnés en cochant la case de la colonne B de la ligne de programme désirée.

Pour chacun de ces modes, vous aurez une information sur le cycle courant, et vous aurez aussi une mise à jour des registres, du pipeline et de la mémoire de données.

III – Prise en main du simulateur

L'objectif de cette première partie est de vous familiariser avec le simulateur, en revenant sur les notions du cours et celles de l'assembleur.

Dans un premier temps, on travaillera sur un programme simple : la suite de Fibonacci. Les codes C et assembleur sont les suivants :

```
.data 0
.program 100

    li R0, 0          -- Inst1
    si (R0), R0       -- Inst2
    add R0, R0, 1     -- Inst3
    si (R0), R0       -- Inst4
    add R0, R0, 1     -- Inst5
    li R1, 1          -- Inst6
    li R2, 0          -- Inst7
    li R3, 1022       -- Inst8

.boucle
    li R4, (R1)       -- Inst9
    li R5, (R2)       -- Inst10
    add R6, R4, R5    -- Inst11
    si (R0), R6       -- Inst12
    add R0, R0, 1     -- Inst13
    add R1, R1, 1     -- Inst14
    add R2, R2, 1     -- Inst15
    sub R3, R3, 1     -- Inst16
    brnz R3, boucle   -- Inst17
    exit              -- Inst18

.end
```

Listing 1 – Code assembleur de la suite pour l'outil JSimRisc

```
void main()
{
    int i, n;
    int u[1022],

    u[0] = 1;
    u[1] = 1;
    n = 1022;

    for (i = 2 ; i < n ; i++)
    {
        u[i] = u[i-2] + u[i-1];
    }
}
```

Listing 2 – Code C de la suite

IV - Liste des instructions assembleur

A titre d'information, on donne ci-dessous le tableau de l'ensemble des instructions du processeur dont on peut réaliser la simulation au travers de l'interface JSimRisc .

Type d'instructions	Signification
Transfert de données registres/mémoire	
li, si	chargement/rangement mot vers/depuis des registres entiers
lf, sf	chargement/rangement flottant simple précision
move	transfert entre registres
cmovz, cmovnz, cmovn, cmovp	transfert conditionnel entre registres
Arithmétique et logique	
add, addf	addition sur des entiers et flottants
sub, subf	soustraction sur des entiers et flottants
mult, multf	multiplication sur des entiers et flottants
and, nand	et, non et
or, nor, xor	ou, non ou, et ou exclusif
Contrôle	
br	branchement inconditionnel
brz, brnz	branchement si registre égal/non égal à zéro
brp, brn	branchement si registre positif/négatif
nop	instruction vide
exit	fin du programme

TABLE 2 – Instructions de l'architecture JSimRisc